

LABORATORY OBSERVATION

Course: Full Stack Web Development

Course Code: 23CM4121

Branch: CSM

Section: B

Assignment:01

Student Name: K. Saranya

Roll No.: A24126552090

1. TITLE

Design and Develop JavaScript Programs to Demonstrate the Different Types of Inheritance — Single, Multilevel, Hierarchical, Multiple, and Hybrid — Using ES6 Classes and Mixins.

2. AIM

To design and implement JavaScript programs that demonstrate the five types of inheritance — single, multilevel, hierarchical, multiple, and hybrid — using ES6 class syntax, the extends keyword, and prototype-based mixins for simulating multiple inheritance.

3. OBJECTIVE

The objectives of this experiment are:

- To understand the concept of inheritance in object-oriented JavaScript.
- To implement single inheritance using the class and extends keywords.
- To implement multilevel inheritance by chaining classes across multiple levels.
- To implement hierarchical inheritance where multiple child classes extend one parent class.
- To simulate multiple inheritance in JavaScript using mixins and Object.assign().
- To implement hybrid inheritance by combining class-based inheritance with mixins.
- To create and instantiate objects, and invoke inherited and own methods correctly.

4. THEORY

Inheritance is an object-oriented programming concept that allows one class (child) to acquire the properties and methods of another class (parent), promoting code reuse. JavaScript implements inheritance through a prototype chain, and ES6 introduced the class and extends syntax as syntactic sugar over this prototype-based model.

The following types of inheritance are demonstrated in this program:

- Single Inheritance — one child class (Customer) extends exactly one parent class (User), inheriting its methods directly.
- Multilevel Inheritance — a class (PremiumCustomer) extends a class (Customer2) that itself extends another class (User2), forming a chain of inheritance across multiple levels.
- Hierarchical Inheritance — more than one child class (Customer3 and Admin) independently extends the same parent class (User3).

- Multiple Inheritance — since JavaScript classes support only a single extends clause, multiple inheritance is simulated using mixins (plain objects holding methods) copied onto a class's prototype with Object.assign().
- Hybrid Inheritance — a combination of class-based inheritance and mixins, where a class inherits from a parent via extends and additionally gains methods from a mixin object.

The basic flow of the program is:

Class definitions are declared → Child classes extend parent classes (or mixins are merged into a prototype) → Objects are instantiated with the new keyword → Inherited and own methods are invoked on the object.

5. ALGORITHM / PROCEDURE

1. Create a new JavaScript file named inheritance.js.
2. Define a base class User with a login() method to demonstrate single inheritance.
3. Define a class Customer that extends User and adds a buyProduct() method; instantiate it and call both methods.
4. For multilevel inheritance, define User2 → Customer2 (extends User2) → PremiumCustomer (extends Customer2), each adding a new method.
5. Instantiate PremiumCustomer and call the login(), buyProduct(), and getDiscount() methods to confirm the full chain works.
6. For hierarchical inheritance, define User3 as the parent, and Customer3 and Admin as two separate classes that both extend User3.
7. Instantiate both Customer3 and Admin and call their respective inherited and own methods.
8. For multiple inheritance, define two mixin objects, Login and Payment, each holding one method.
9. Define a class Employee with its own work() method, then use Object.assign(Employee.prototype, Login, Payment) to merge both mixins into it.
10. Instantiate Employee and call work(), login(), and makePayment() to verify all three sources of behaviour.
11. For hybrid inheritance, define User4 as a parent class, with Customer4 and Admin4 extending it via class-based inheritance.
12. Define a Notification mixin and merge it into Customer4.prototype using Object.assign(), combining inheritance with a mixin.
13. Instantiate Customer4 and Admin4 and call their inherited, own, and mixed-in methods.
14. Run the file using Node.js (node inheritance.js) and verify the console output for each section.

6. PROGRAM

inheritance.js

```

1  // 1. SINGLE INHERITANCE
2  console.log("Single Inheritance");
3
4  class User {
5      login() {
6          console.log("User is logging in.");
7      }
8  }
9
10 class Customer extends User {
11     buyProduct() {
12         console.log("Customer is buying a product.");

```

```
13     }
14 }
15
16 let customer = new Customer();
17 customer.login();
18 customer.buyProduct();
19
20
21 // 2. MULTILEVEL INHERITANCE
22 console.log("\nMultilevel Inheritance");
23
24 class User2 {
25     login() {
26         console.log("User is logging in.");
27     }
28 }
29
30 class Customer2 extends User2 {
31     buyProduct() {
32         console.log("Customer is buying a product.");
33     }
34 }
35
36 class PremiumCustomer extends Customer2 {
37     getDiscount() {
38         console.log("Premium customer gets a discount.");
39     }
40 }
41
42 let premiumCustomer = new PremiumCustomer();
43 premiumCustomer.login();
44 premiumCustomer.buyProduct();
45 premiumCustomer.getDiscount();
46
47
48 // 3. HIERARCHICAL INHERITANCE
49 console.log("\nHierarchical Inheritance");
50
51 class User3 {
52     login() {
53         console.log("User is logging in.");
54     }
55 }
56
57 class Customer3 extends User3 {
58     buyProduct() {
59         console.log("Customer is buying a product.");
60     }
61 }
62
63 class Admin extends User3 {
64     manageUsers() {
65         console.log("Admin is managing users.");
66     }
67 }
68
69 let customer3 = new Customer3();
70 let admin = new Admin();
71
72 customer3.login();
73 customer3.buyProduct();
74
75 admin.login();
76 admin.manageUsers();
```

```

77
78
79 // 4. MULTIPLE INHERITANCE
80 // Using Mixins
81 console.log("\nMultiple Inheritance");
82
83 const Login = {
84     login() {
85         console.log("Employee is logging in.");
86     }
87 };
88
89 const Payment = {
90     makePayment() {
91         console.log("Employee is making payment.");
92     }
93 };
94
95 class Employee {
96     work() {
97         console.log("Employee is working.");
98     }
99 }
100
101 // Add both Login and Payment features
102 Object.assign(Employee.prototype, Login, Payment);
103
104 let employee = new Employee();
105 employee.work();
106 employee.login();
107 employee.makePayment();
108
109
110 // 5. HYBRID INHERITANCE
111 console.log("\nHybrid Inheritance");
112
113 class User4 {
114     login() {
115         console.log("User is logging in.");
116     }
117 }
118
119 class Customer4 extends User4 {
120     buyProduct() {
121         console.log("Customer is buying a product.");
122     }
123 }
124
125 class Admin4 extends User4 {
126     manageUsers() {
127         console.log("Admin is managing users.");
128     }
129 }
130
131 const Notification = {
132     sendNotification() {
133         console.log("Notification sent.");
134     }
135 };
136
137 // Add Notification feature to Customer
138 Object.assign(Customer4.prototype, Notification);
139
140 let customer4 = new Customer4();

```

```

141 let admin4 = new Admin4();
142
143 customer4.login();
144 customer4.buyProduct();
145 customer4.sendNotification();
146
147 admin4.login();
148 admin4.manageUsers();

```

7. CODE EXPLANATION

Keyword / Concept	Description
class	Declares a blueprint for creating objects with shared methods and behaviour.
extends	Establishes single/multilevel/hierarchical inheritance by linking a child class's prototype to a parent class.
new ClassName()	Creates an instance (object) of a class, giving it access to its own and all inherited methods.
Multilevel chain (A → B → C)	Each class extends the one before it, so the last class in the chain inherits methods from every ancestor.
Hierarchical structure	Two or more classes independently extend the same parent, each gaining the parent's methods separately.
Mixin object (e.g., Login, Payment)	A plain object holding reusable methods, used to add behaviour to a class without using extends.
Object.assign()	Copies methods from one or more mixin objects onto a class's prototype, simulating multiple inheritance.
Hybrid combination	Uses extends for the class hierarchy and Object.assign() for mixins together in the same class.
console.log()	Prints output to the console, used here to trace which method (inherited or own) is being executed.

8. TEST CASES AND EXECUTION DATA

The script was run using Node.js and the console output for each section was recorded below.

Test Case: TC-01 — Single Inheritance

Action	Create a Customer object and call login() and buyProduct().
Expected Result	Customer should access its own method and the inherited login() method from User.
Actual Result	Output: "User is logging in." followed by "Customer is buying a product."
Status	Pass

```

PS C:\Users\SARANYA\OneDrive\Documents\fullstackwebdevelopment\04-assignments\code> node assignment1_inheritance.js
Single Inheritance
User is logging in.
Customer is buying a product.

```

Test Case: TC-02 — Multilevel Inheritance

Action	Create a PremiumCustomer object and call login(), buyProduct(), and getDiscount().
Expected Result	Methods from all three levels of the chain (User2 → Customer2 → PremiumCustomer) should execute.
Actual Result	Output: "User is logging in.", "Customer is buying a product.", "Premium customer gets a discount."
Status	Pass

```
Multilevel Inheritance
User is logging in.
Customer is buying a product.
Premium customer gets a discount.
```

Test Case: TC-03 — Hierarchical Inheritance

Action	Create both a Customer3 object and an Admin object from the same parent User3.
Expected Result	Both objects should independently access the inherited login() method along with their own distinct method.
Actual Result	Customer3 printed "User is logging in." and "Customer is buying a product."; Admin printed "User is logging in." and "Admin is managing users."
Status	Pass

```
Hierarchical Inheritance
User is logging in.
Customer is buying a product.
User is logging in.
Admin is managing users.
```

Test

Case: TC-04 — Multiple Inheritance (Mixins)

Action	Merge the Login and Payment mixins into Employee.prototype and call work(), login(), makePayment().
Expected Result	Employee should access its own method plus both mixed-in methods, simulating inheritance from two sources.
Actual Result	Output: "Employee is working.", "Employee is logging in.", "Employee is making payment."
Status	Pass

```
Multiple Inheritance
Employee is working.
Employee is logging in.
Employee is making payment.
```

Test Case: TC-05 — Hybrid

Inheritance

Action	Create Customer4 (with the Notification mixin merged in) and Admin4, both extending User4.
--------	--

Expected Result	Customer4 should show inherited, own, and mixed-in behaviour; Admin4 should show only inherited and own behaviour.
Actual Result	Customer4 printed "User is logging in.", "Customer is buying a product.", "Notification sent."; Admin4 printed "User is logging in." and "Admin is managing users."
Status	Pass

```

Hybrid Inheritance
User is logging in.
Customer is buying a product.
Notification sent.
User is logging in.
Admin is managing users.

```

9. OUTPUT

Console Output:

```

PS C:\Users\SARANYA\OneDrive\Documents\fullstackwebdevelopment\04-assignments\code> node assignment1_inheritance.js
Single Inheritance
User is logging in.
Customer is buying a product.

Multilevel Inheritance
User is logging in.
Customer is buying a product.
Premium customer gets a discount.

Hierarchical Inheritance
User is logging in.
Customer is buying a product.
User is logging in.
Admin is managing users.

Multiple Inheritance
Employee is working.
Employee is logging in.
Employee is making payment.

Hybrid Inheritance
User is logging in.
Customer is buying a product.
Notification sent.
User is logging in.
Admin is managing users.

```

10. RESULT

Thus, JavaScript programs demonstrating single, multilevel, hierarchical, multiple, and hybrid inheritance were successfully designed, implemented, and executed. ES6 class syntax with the extends keyword was used for class-based inheritance, while mixins combined with Object.assign() were used to simulate multiple inheritance, and all outputs were verified to match the expected results.